

SIH26171 Privacy-Preserving Vision Agent

A browser extension that reads a webpage, detects and hides personal information on the user device, asks an AI planner what to do, and performs one safe browser action at a time.

Important distinction: your project uses two different AI components. **YuNet** is the local ONNX computer-vision detector inside the extension. **Qwen2.5-VL:3B** is the server-side vision-language model that plans the next task action. They have different jobs.

1. The Project In One Picture

1. User gesture

Toolbar click opens the panel and attaches the active tab.

2. Page snapshot

Content script clones HTML and records real element geometry.

3. Capture

Browser captures the visible tab as an image.

4. Local detection

DOM rules find PII; YuNet can find faces.

5. Local redaction

Text is replaced and image pixels are covered.

6. Planning

Only sanitized context reaches the local planner or Qwen server.

7. Validation

Client checks the returned action and its target ref.

8. Execution

Content script resolves the private DOM path and clicks/types.

9. Repeat

The orchestrator repeats until done, stopped, or blocked.

Toolbar click

- > background.ts
- > content.ts: snapshot clone + viewport
- > captureVisibleTab
- > local YuNet + DOM PII scan
- > merge detections
- > HTML redact + pixel bake
- > SanitizedContext
- > local-planner OR HTTP /plan -> Qwen2.5-VL
- > parseAction + validateAction
- > private ref -> DOM path
- > execution/actions.ts
- > next step

2. Folder And File Structure

This is the practical map. Files under `src/` are the extension library; `server/` is a separate Node process and is deliberately outside the extension bundle.

Path	Purpose
<u>README.md</u>	Short overview, setup, layout, and important pure functions.
<u>CLAUDE.md</u>	Authoritative architecture, security constraints, measurements, known gaps, and decisions about how the project must be changed.
<u>DECISIONS.md</u>	Measured experiments and engineering choices, such as model selection and latency findings.
<u>package.json</u> / package-lock.json	Dependencies, npm commands, Node version requirement, and locked versions.
<u>tsconfig.json</u>	Strict TypeScript compiler rules and path aliases such as <code>@/contracts</code> .
<u>wxt.config.ts</u>	Builds Chrome/Firefox Manifest V3 packages, permissions, icons, CSP, and runtime content-script policy.
<u>public/</u>	Packaged icons, model weights under <code>models/</code> , and ONNX Runtime Web files under <code>wasm/</code> .
<u>server/</u>	Node HTTP process: reads environment variables, chooses a planner, and exposes HTTP endpoints.
<u>spike/</u>	Earlier throwaway Transformers.js/offscreen experiments and measurements. It is research material, not the main extension path.
<u>test-site/</u>	Offline local website and scripts for testing real browser behavior and vision images.

<u>tests/</u>	Unit, integration, architecture, built-manifest, security, scoring, and type-level tests.
<u>scripts/</u>	Model vending, icon creation, and PDF generation helpers.
<code>.output/</code> , <code>.wxt/</code> , <code>node_modules/</code>	Generated build output, WXT generated files, and installed dependencies. Do not treat these as source.
Root <code>*.onnx</code> , <code>*.cc</code> , probes, <code>ear.b64</code>	Model experiments and low-level probes. The currently packaged extension vision path is YuNet under <code>public/models/opencv/</code> .

The source modules

contracts/

Shared types and pure rules. Includes `action.ts` , `context.ts` , `untrusted.ts` , geometry, image, detection, redaction, vision, messages, budgets, and metrics.

perception/

Screen capture, inference hosts, ONNX runtime, YuNet decoding, resizing, post-processing, and model benchmarking.

redaction/

DOM PII scanning, detection merging, HTML masking, pixel operations, geometry stamping, sanitization, and browser-specific DOM pipelines.

agent-server/

Action protocol, prompt creation, parser, validator, local planner, HTTP client, origin permissions, and server planner implementations.

orchestrator/

The exact step pipeline and bounded repeated loop.

execution/

Performs already-validated click, type, select, scroll, keyboard, navigation, wait, and terminal actions.

panel/

Pure state reducer, selectors, React/Preact UI components, and metrics/debug display.

entrypoints/

Thin browser adapters: background, content script, Chrome offscreen page, and side panel.

harness/

Fixtures, scoring, timing, resource budgets, and benchmark reports. Node-only.

3. Browser Extension Entry Points

[src/entrypoints/background.ts](#)

This is the coordinator. It listens to the toolbar action, records the tab that received the user gesture, opens Chrome's side panel or Firefox's sidebar, creates the correct inference host, communicates with the content script, starts one step or the loop, and emits events to the panel.

Chrome runs this as a Manifest V3 service worker. A service worker has no DOM, canvas, or WebGPU, so it cannot host the model. Firefox uses an MV3 background event page, which has a DOM. This difference is why `perception/host/` has separate Chrome and Firefox implementations.

[src/entrypoints/content.ts](#)

This is the only code that touches the webpage DOM. It clones the page rather than modifying the live page, stamps real `getBoundingClientRect()` geometry into the clone, and returns the HTML plus viewport information. It also receives an already validated action and executes it. The content script never talks to Qwen and never decides what to click.

[src/entrypoints/offscreen/](#)

Chrome-only offscreen document. It owns DOM parsing, ONNX Runtime, YuNet inference, frame retention, pixel baking, and browser operations that cannot run in the service worker.

[src/entrypoints/sidepanel/](#)

Mounts the panel UI and sends user commands/events. The panel is a view of real runtime state; it is not the agent brain.

4. The Complete Pipeline

4.1 User starts the agent

The toolbar button is declared by `wxt.config.ts`. There is intentionally no popup. The click must remain a synchronous user gesture because it grants temporary `activeTab` access and allows the panel to open. The content script is runtime-registered, so the extension does not automatically read every website after installation.

4.2 Snapshot and geometry

The content script makes a clone of `document.documentElement`. The original page is not modified. Each important element receives a stable-looking reference such as `e7` in the serialized context. The clone includes layout rectangles from the real page so that redaction can line up with screenshot pixels later.

4.3 Capture

`captureVisibleTab` captures the visible tab for the window. It is not a general tab selector, so the attached tab must still be the active tab. Capture is throttled because browsers limit it to about two calls per second.

4.4 DOM detection

`dom-scan.ts` uses three levels of evidence:

Level	Example	Meaning
Structural truth	<code><input type="password"></code>	The browser element itself identifies the sensitive type.
Declared intent	<code>autocomplete="cc-number"</code>	The website declares what information the field expects.
Heuristics	label, id, placeholder, text keywords	A lower-confidence guess, such as “CVV” or “date of birth”.

It also calculates accessible names and roles, reads control values correctly, strips forged placeholder-shaped tokens, and creates a canonical DOM path used only locally.

4.5 Local vision detection

The shipped extension model is OpenCV YuNet, loaded as an ONNX file by [**yunet-backend.ts**](#). It detects faces. It does not understand a task and it does not choose clicks. Its result is a list of visual bounding boxes, which is merged with DOM detections.

4.6 Merge detections

[**merge.ts**](#) combines DOM and vision results. Corroborating evidence can raise confidence, and duplicate detections are removed. The result identifies what needs to be hidden and where it is located.

4.7 Redact HTML

[**redact.ts**](#) parses the clone, strips page-forged redaction tokens, and applies strategies from [**strategies.ts**](#). Depending on the detection, it can replace a value with a token such as `[[PII:EMAIL:1:nonce]]`, remove a node, mask an attribute, or drop a sensitive value.

4.8 Redact image pixels

HTML redaction cannot hide pixels already present in a screenshot. Pixel operations are created by the redaction layer and applied by [**canvas-redact.ts**](#) to the retained RGBA frame. The image is not sent when a visible required redaction could not be covered. Off-screen detections are counted separately because they are not visible in the captured viewport.

4.9 Build SanitizedContext

[**sanitize.ts**](#) creates the only object allowed to leave the privacy pipeline. It contains bounded page structure, safe element names/roles/states, redaction summary, nonce, and optionally a baked screenshot. It does not contain raw PII values, raw DOM paths, or the unredacted frame.

5. The Two AI Models

5.1 YuNet: the extension's local vision model

YuNet is an object detector specialized for faces. The flow is:

1. Decode the screenshot into RGBA pixels.
2. Resize and letterbox it to the configured square model size.
3. Convert RGBA to BGR, CHW layout, float values in the raw 0-255 range.
4. Run ONNX Runtime Web using WebGPU if requested, otherwise WASM.
5. Decode the model's stride 8, 16, and 32 output heads.
6. Undo letterboxing and convert boxes back to CSS viewport coordinates.
7. Apply confidence filtering and non-maximum suppression.

Measured project notes report roughly 232,589 model bytes and about 30 ms inference in the WASM benchmark. Older YOLOS experiments were much larger and slower, so the active path is YuNet.

What YuNet cannot do: it is not Qwen. It cannot read the user goal, understand a form, or decide “click Add to cart.” Its job is only visual evidence, currently faces. DOM rules handle most supported PII kinds such as email, phone, password, card fields, address, OTP, and identity-number labels.

5.2 Qwen2.5-VL:3B: the server-side planner

Qwen2.5-VL is a vision-language model. It receives a prompt plus, when safe, a redacted screenshot. It reads the element list and user goal, reasons about which element should be acted on, and returns exactly one JSON action. It is not given direct browser access.

The integration is in [vlm-planner.ts](#). It uses the OpenAI-compatible chat-completions request shape, which Ollama, vLLM, llama.cpp server, and several hosted providers support.

```
{
  "model": "qwen2.5vl:3b",
  "temperature": 0,
  "max_tokens": 160,
  "messages": [{
```

```
"role": "user",
"content": [
  {"type": "image_url", "image_url": {"url": "data:image/jpeg;base64,..."}},
  {"type": "text", "text": "...sanitized prompt..."}
]
}]
}
```

The screenshot is deliberately sent before the text. The final instruction tells Qwen to return one JSON object and nothing else. The adapter returns the assistant text raw; the client parses and validates it because the server is not trusted.

6. How Qwen Reads The Screenshot And Does A Task

1. **Visual encoding:** Qwen's vision component converts image patches into visual features. It can associate visible controls, labels, cards, and layout with the textual prompt.
2. **Prompt grounding:** the text prompt supplies the user goal, viewport, redaction scheme, and a bounded list such as `ref=e7 role=button name="Add to cart"`.
3. **Candidate choice:** Qwen compares the goal with names, roles, states, positions, and image evidence. It does not receive a CSS selector or DOM path.
4. **Structured response:** it returns an object such as `{"type":"click","ref":"e7"}` or `{"type":"type","ref":"e3","text":"laptop","submit":true}`.
5. **Client gate:** `parseAction` checks the format. `validateAction` checks safety and whether `e7` was actually included in the context.
6. **Execution:** the background looks up the private path corresponding to `e7`; the content script resolves that path in the live page and performs the action.

Qwen does not click by itself. It only proposes a structured action. Browser extension code performs the click after validation. This separation is what prevents a model response from becoming arbitrary browser control.

7. How Automatic Clicking Works

The model sees a reference, not a selector. During snapshot processing the extension walks the document and associates each exposed element with a private DOM path, for example a path made from `button:nth-of-type(2)`. That path stays in the extension.

```
Qwen output: {"type":"click","ref":"e7"}
```

```
background:
```

```
  e7 -> private DOM path
```

```
  sends action + path to content script
```

```
content script:
```

```
  resolveDomPath(document, path)
```

```
  element.click()
```

For typing, the executor finds the input/textarea/select, assigns its live `.value``, dispatches input and change events, and optionally submits the form with `requestSubmit()`. For a select action it matches the requested option by value or visible label. For navigation, the URL was already origin-checked.

If the page changed and the path no longer resolves, execution reports a miss. It does not guess another element. This is safer than silently clicking the wrong control.

8. How Privacy Redaction Stops Server Leakage

Text path

Page-derived strings start as untrusted data. The code converts them to a controlled `DataAtom` only after removing control characters, bidirectional/zero-width tricks, prompt-fence tokens, and excessive length. The prompt renderer accepts page strings only through `DataAtom`. They are placed inside explicit page-data fence markers and are described as inert data, never instructions.

Screenshot path

The screenshot is captured before redaction, but it is retained locally. Detection produces pixel operations. The image is baked locally; the raw frame is released after the step. If a visible sensitive region cannot be covered, the image is omitted rather than leaked.

Action path

The server receives only a ref, not a DOM path. The client checks that the ref was sent, rejects sensitive typing without user confirmation, checks URL origins, and limits scroll, wait, and text size. A server cannot name an element that was not exposed.

Protection	What it prevents
PII replacement tokens	Qwen sees that a sensitive value exists, but not the value.
Pixel baking	Qwen cannot read PII from the screenshot.
Untrusted/DataAtom types	Page text cannot accidentally be treated as an instruction.
Nonce	A hostile page cannot forge a real-looking redaction token.

Ref allowlist	The server cannot target an unexposed DOM element.
Origin allowlist	Navigation cannot go to an arbitrary origin.
Credential question rejection	A compromised planner cannot ask for a password, OTP, CVV, or card number in the extension UI.

9. Server Architecture

server/main.ts

This starts the Node process. It reads `PORT`, `VLM_ENDPOINT`, `VLM_MODEL`, and optional `VLM_API_KEY`. If endpoint or model is missing, it selects `HeuristicPlanner` and does not contact a network. If both exist, it creates `VlmPlanner`.

server/agent-http.ts

This exposes the HTTP server and routes requests such as `POST /plan` and `GET /health` to the server application.

src/agent-server/server/app.ts

This validates protocol version, request size, nonce consistency, and planner output handling. It invokes the selected planner but does not execute browser actions.

src/agent-server/client.ts

This is the extension-side HTTP client. It sends a sanitized context to the configured server and handles timeout/network errors.

src/agent-server/local-planner.ts

This is the offline baseline. It can perform simple text-entry and name-overlap click tasks without Qwen. It is useful for testing the whole pipeline without a server.

10. Action Parsing And Validation

[contracts/action.ts](#) defines the legal vocabulary: `click`, `type`, `select`, `scroll`, `key`, `navigate`, `wait`, `ask_user`, `done`, and `abort`.

[parse-action.ts](#) extracts one JSON object from raw Qwen output and rejects malformed, missing, extra, or wrongly typed fields.

[validate-action.ts](#) is the runtime security backstop:

- `unknown-ref`: ref was not in the sent element list.
- `sensitive-target`: typing into PII requires explicit user confirmation.
- `not-typeable`: model tried to type into a button or link.
- `origin-not-allowed` / `bad-url`: unsafe navigation.
- `scroll-too-far`, `wait-too-long`, `text-too-long`: resource and abuse limits.
- `asks-for-credential`: the planner question solicits a password, card number, code, or similar secret.

11. Orchestrator And Loop

[step.ts](#) composes one complete iteration in privacy-preserving order: snapshot, capture, detect, redact, bake, sanitize, plan, parse, validate, execute. Dependencies are injected so this can be tested outside a browser.

[loop.ts](#) repeats steps, records action history, detects no-progress/repeated actions, supports cancellation, and stops at a maximum of eight steps. Vision failures degrade to DOM-only operation after a breaker threshold rather than destroying the entire task.

Possible terminal outcomes include `done`, `abort`, `ask_user`, error, cancellation, no progress, or maximum steps.

12. Panel, Harness, And Tests

[panel/state.ts](#) is a pure reducer from panel events to panel state. The UI can therefore be replaced without changing the agent contracts. It displays host/model status, redaction counts, timing, errors, planner output, and task status.

[src/harness/](#) contains fixtures and score functions. Fixture triples use an HTML page, truth JSON, and plausible vision JSON. Metrics cover screen context, PII detection, redaction, resources, and latency.

The tests are organized by responsibility:

Test folder	What it proves
tests/contracts	Types, budgets, ambiguity, vendored model paths.
tests/architecture	Module dependency boundaries, forbidden imports, manifest/security rules.
tests/perception	Capture, hosts, dispatch, letterboxing, YuNet, runtime and post-processing.
tests/redaction	DOM scanning, merging, HTML/pixel redaction, geometry, leak guards.
tests/agent-server	Prompt, parser, validator, clients, origin, planners, VLM adapter.
tests/orchestrator	Step order, failure handling, loop bounds and progress.
tests/execution	Real DOM behavior for clicks, typing, selection, scrolling and keys.
tests/built	Emitted Chrome/Firefox manifests, bundle contents, and panel startup.

tests/types	Negative TypeScript tests proving unsafe untrusted-data code does not compile.
-------------	--

13. Running The Project

```
npm install
npm test
npm run build
npm run build:chrome
npm run build:firefox
npm run test:built
npm run scorecard
```

Run Qwen through Ollama

```
npm run ollama:serve
npm run ollama:pull
$env:VLM_ENDPOINT = "http://localhost:11434/v1/chat/completions"
$env:VLM_MODEL = "qwen2.5v1:3b"
npm run server
```

The server should print that it is listening on port 8787 and identify the selected planner. In the extension panel, supply/grant the server origin, load the local model if enabled, attach a tab, and run a task.

Run the local test site

```
npm run test-site
npm run test-site:verify
npm run test-site:images
npm run test-site:vision
```

Build output is placed under `.output/chrome-mv3/` or the Firefox equivalent and can be loaded as an unpacked extension.

14. Current Limitations And Honest Expectations

- YuNet detects faces; it is not a general PII detector. Names in free prose, signatures, cards, and ID documents depend on DOM rules or future model work.
- The default planner may be the network-free local planner unless both Qwen environment variables are set.
- Qwen is probabilistic and can return malformed JSON or select the wrong action; parser, allowlists, budgets, and bounded retries contain that risk but do not make the model perfect.
- Two local models on one GPU can compete for memory. Qwen planning and browser vision may become slow when run together.
- Active-tab access is temporary. Navigation can remove it; multi-step work needs the user-granted site permission path.
- Refs are positional per snapshot. A changed page can make an old ref stale, which should be reported as a miss rather than guessed.
- Real browser verification, model weights, Ollama, and permissions are runtime prerequisites. Green unit tests do not mean every deployment environment has a working model.

15. Beginner Glossary

Term	Meaning here
DOM	The browser's tree of HTML elements: buttons, inputs, headings, forms, and so on.
Content script	Extension code injected into a webpage that can read and interact with that page's DOM.
Background service worker	Chrome's event-driven extension coordinator. It has no page DOM.
Offscreen document	Hidden Chrome extension page used here for DOM parsing, canvas/image work, and model inference.
ONNX	A portable neural-network model format.
Inference	Running a trained model on input data to produce a prediction.
VLM	Vision-language model: an AI model that can reason over text and images.
PII	Personally identifiable information, such as email, phone, password, card, address, or identity number.
Prompt	The instructions and data sent to the language model.
Ref	A short model-visible element identifier such as <code>e7</code> .
Allowlist	A list of things explicitly permitted. Anything outside it is rejected.
Manifest V3	The current browser extension platform format used by Chrome and Firefox here.

WXT	The build framework that turns the TypeScript entrypoints into browser extension bundles.
-----	---

16. The Best Mental Model

contracts = vocabulary

Types and legal shapes shared by every module.

perception = eyes

Screenshot capture and local visual detection.

redaction = privacy filter

Finds and hides sensitive information.

Qwen = planner

Chooses one next action from safe context.

validation = security gate

Rejects unsafe or impossible model requests.

execution = hands

Performs the approved browser operation.

orchestrator = controller

Repeats the process until the task ends.

The model is not given the keys to the browser. It is shown a carefully limited description of the page and proposes one action. The extension owns privacy, validation, element resolution, and execution.

Generated for the SIH26171 workspace. This guide explains the implemented architecture and should be read together with [CLAUDE.md](#) and [DECISIONS.md](#) for the latest measured status.